97. Передача по значению

По умолчанию, аргументы в C++ передаются по значению. Когда аргумент **передается по значению**, то его значение копируется в параметр функции. Например:

```
#include <iostream>

void boo(int y)
{
    std::cout << "y = " << y << std::endl;
}

int main()
{
    boo(7); // 1-й вызов

    int x = 8;
    boo(x); // 2-й вызов
    boo(x + 2); // 3-й вызов

    return 0;
}
```

В первом вызове функции boo() аргументом является **литерал** `7`. При вызове boo() создается переменная `y`, в которую копируется значение `7`. Затем, когда boo() завершает свое выполнение, переменная `y` уничтожается.

Во втором вызове функции boo() аргументом является уже переменная `x = 8`. Когда boo() вызывается во второй раз, переменная `y` создается снова и значение `8` копируется в `y`. Затем, когда boo() завершает свое выполнение, переменная `y` снова уничтожается.

В третьем вызове функции boo() аргументом является выражение `x + 2`, которое вычисляется в значение `10`. Затем это значение передается в переменную `y`. При завершении выполнения функции boo() переменная `y` вновь уничтожается.

Таким образом, результат выполнения программы:

```
y = 7
y = 8
y = 10
```

Поскольку в функцию передается копия аргумента, то исходное значение не может быть изменено функцией. Это хорошо проиллюстрировано в следующем примере:

```
#include <iostream>

void boo(int y)
{
    std::cout << "y = " << y << '\n';

    y = 8;

    std::cout << "y = " << y << '\n';
} // y уничтожается здесь

int main()
{
    int x = 7;
    std::cout << "x = " << x << '\n';

    boo(x);

    std::cout << "x = " << x << '\n';
    return 0;
}
```

Результат:

```
x = 7
y = 7
y = 8
x = 7
```

В начале функции main() `x` равно `7`. При вызове boo() значение `x` (`7`) передается в параметр `y` функции boo(). Внутри boo() переменной `y` сначала присваивается значение `8`, а затем `y` уничтожается. Значение `x` не изменяется, даже если изменить `y`.

Параметры функции, переданные по значению, также могут быть **const**. Тогда уже будет 100% гарантия того, что функция не изменит значение параметра.

Плюсы и минусы передачи по значению

Плюсы передачи по значению:

+ Аргументы, переданные по значению, могут быть переменными (например, `x`), литералами (например, `8`), выражениями (например, `x + 2`), **структурами**, классами или **перечислителями** (т.е. почти всем, чем угодно).
+ Аргументы никогда не изменяются функцией, в которую передаются, что предотвращает возникновение **побочных эффектов**.

Минусы передачи по значению:

− Копирование структур и классов может привести к значительному снижению производительности (особенно, когда функция вызывается много раз).

Когда использовать передачу по значению:

→ При передаче фундаментальных типов данных и перечислителей, когда предполагается, что функция не должна изменять аргумент.

Когда не использовать передачу по значению:

→ При передаче **массивов**, структур и классов.

В большинстве случаев, передача по значению — это наилучший способ передачи аргументов фундаментальных типов данных, когда функция не должна изменять исходные значения. Передача по значению является гибкой и безопасной, а в случае фундаментальных типов данных еще и эффективной.